

Rapport d'Évaluation de la Sécurité des APIs – HealthMate

Application : HealthMate

Architecture : Flutter (Mobile) – Laravel (REST API)

Version : 1.1

Classification : Interne – Sécurité Applicative

Rédigé par : safia Massaid

Date du rapport : 09 février 2026

1. Objectif et périmètre

Ce document présente une évaluation exhaustive et structurée de la posture de sécurité des APIs REST qui constituent la colonne vertébrale technologique de l'écosystème HealthMate. Dans un secteur aussi critique que celui de la e-santé, où la confiance numérique est le garant de l'adoption par les patients et les praticiens, cet audit vise à établir un diagnostic précis des risques techniques, logiques et organisationnels. L'objectif fondamental est d'identifier, de quantifier et de hiérarchiser toute vulnérabilité susceptible de compromettre les trois piliers de la sécurité de l'information (CIA) : la **confidentialité** absolue des données médicales protégées, l'**intégrité** irréprochable des transactions applicatives (telles que la prise de rendez-vous ou l'historique des soins), et la **disponibilité** continue du service face à d'éventuelles tentatives de déni de service ou de pannes systémiques.

Le périmètre de cette analyse s'étend sur l'ensemble de la chaîne de valeur technique, allant des mécanismes d'authentification et de gestion de session via JWT jusqu'au contrôle d'accès granulaire aux ressources. Une attention particulière est portée à la gestion des données sensibles au repos et en transit, ainsi qu'à la robustesse des flux réseau et des intégrations tierces, notamment avec Firebase Cloud Messaging. Au-delà de l'aspect purement technique, ce rapport évalue l'alignement des pratiques de développement avec la méthodologie **DevSecOps** et vérifie la conformité rigoureuse avec les cadres réglementaires en vigueur, notamment le **RGPD** pour l'Espace Économique Européen et les standards de type **HDS/HIPAA** pour l'hébergement de données de santé. Cette démarche s'inscrit dans une volonté de protection proactive de la réputation de l'entreprise et de réduction de la surface d'attaque globale de l'application mobile et de son backend Laravel.

2. ARCHITECTURE TECHNIQUE ET SURFACE D'EXPOSITION

2.1 Vue d'ensemble de l'écosystème

L'architecture de la solution HealthMate est conçue selon un modèle de micro-services découplés, favorisant la scalabilité et la maintenance, mais multipliant mécaniquement les points d'entrée potentiels. Le système repose sur les briques technologiques suivantes :

- **Client mobile** : Développé avec le framework **Flutter**, assurant une base de code unifiée pour Android et iOS. La sécurité repose ici sur l'isolation du sandbox applicatif et la gestion sécurisée du stockage local.

- **Backend** : Une API RESTful propulsée par **Laravel**, choisie pour ses fonctionnalités natives de protection (Eloquent ORM, Middleware, Policies).
- **Base de données** : Instance persistante **MySQL ou PostgreSQL**, avec chiffrement des données au repos (At-Rest Encryption) pour les champs identifiés comme sensibles.
- **Services externes** : Intégration de **Firebase Cloud Messaging (FCM)** pour la gestion des notifications push asynchrones.
- **Authentification** : Implémentation du standard **JWT (JSON Web Token)**.
L'architecture est strictement *stateless*, délégrant la validation de l'identité à la vérification de la signature du token à chaque requête.

Cette architecture impose une vigilance accrue sur la gestion des secrets applicatifs, la validation rigoureuse des schémas d'entrée et la réduction drastique de la surface d'exposition des points de terminaison (endpoints).

2.2 Classification et criticité des endpoints

La surface d'attaque est segmentée selon une hiérarchie de criticité, permettant d'orienter les efforts de sécurisation vers les flux manipulant des données de santé ou des accès privilégiés.

Endpoint	Méthode	Fonction principale	Niveau de criticité
/api/login	POST	Authentification & Délivrance de tokens	CRITIQUE
/api/register	POST	Création de comptes et injection de données	ÉLEVÉ
/api/doctors	GET	Consultation du catalogue des praticiens	MODÉRÉ
/api/appointments	CRUD	Gestion du cycle de vie des rendez-vous	CRITIQUE
/api/profile	GET/PUT	Accès et modification des PII (Données Perso)	CRITIQUE
/api/notifications	GET	Récupération des alertes et messages	MODÉRÉ

Les endpoints marqués comme **CRITIQUE** font l'objet d'un audit de code systématique et de tests de pénétration ciblés sur les vulnérabilités de type BOLA (Broken Object Level Authorization) et d'usurpation de privilèges.

3. ANALYSE APPROFONDIE DES RISQUES

3.1 Authentification et gestion des sessions

Constats : L'analyse révèle un risque majeur d'exposition des tokens JWT si ces derniers sont stockés de manière non sécurisée sur le terminal mobile (ex: SharedPreferences ou LocalStorage standard). Par ailleurs, l'absence de contrôle d'accès basé sur les rôles (**RBAC**) systématique sur certains points d'entrée secondaires pourrait permettre des escalades de privilèges horizontales ou verticales.

- **Impacts potentiels :** Usurpation complète de session utilisateur et accès non autorisé aux dossiers médicaux confidentiels.
- **Recommandations :** Migration vers flutter_secure_storage. Mise en place d'une rotation périodique des *refresh tokens* et d'un middleware Laravel centralisé pour le RBAC avec mécanisme de révocation (Blacklist) côté serveur.

3.2 Validation des entrées et prévention des injections

Constats : Bien que l'ORM Eloquent de Laravel limite naturellement les injections SQL, l'audit du code montre que des requêtes brutes (raw queries) pourraient être introduites lors de développements futurs complexes. La normalisation des entrées API n'est actuellement pas assez stricte pour contrer des attaques de type XSS ou injections logiques.

- **Recommandations :** Imposer l'utilisation systématique des FormRequest pour chaque endpoint. Appliquer une sanitisation rigoureuse côté serveur et désactiver toute possibilité de requêtes SQL dynamiques basées sur des entrées non validées.

3.3 Gestion des données sensibles

Risques identifiés : Une exposition excessive de champs non nécessaires dans les réponses JSON (payloads trop verbeux) a été détectée. De plus, les journaux système (logs) risquent d'enregistrer des données sensibles si aucun filtre de masquage n'est actif.

- **Mesures recommandées :** Utilisation impérative des API Resources Laravel pour transformer et limiter les modèles exposés. Masquage des identifiants et données PII dans les logs applicatifs. Implémentation d'un chiffrement de couche applicative pour les données de santé les plus critiques en base de données.

3.4 Sécurité des communications

Risques : Le risque d'attaques de type **Man-In-The-Middle (MITM)** est présent si la validation TLS côté client est permissive ou si l'application accepte des certificats auto-signés en production.

- **Actions recommandées :** Imposer TLS 1.2 minimum (idéalement 1.3). Implémenter le **Certificate Pinning** au sein de l'application Flutter pour s'assurer que le client ne communique qu'avec le serveur authentique. Désactiver les suites cryptographiques obsolètes sur le serveur web.

3.5 Protection contre les abus et attaques automatisées

Risques : Les endpoints de /login et de gestion des rendez-vous sont vulnérables au *brute-force* et au *flooding*, ce qui pourrait entraîner une indisponibilité du service (DoS).

- **Mesures :** Configuration d'un *Rate Limiting* différencié par IP et par compte utilisateur. Surveillance proactive des anomalies via un système de logs centralisé et mise en place d'un bannissement progressif des adresses IP suspectes.

3.6 Intégrations tierces (Firebase)

Points d'attention : La fuite de clés serveurs FCM ou la mauvaise gestion des tokens de notification pourrait permettre l'envoi de messages frauduleux aux patients.

- **Recommandations :** Rotation régulière des clés secrètes Firebase. Validation systématique côté serveur de la légitimité du destinataire avant chaque envoi de notification push.

4. CONFORMITÉ ET GOUVERNANCE

4.1 Règlement Général sur la Protection des Données (RGPD)

- L'application QuickCare traitant des données de résidents européens, elle est soumise aux exigences strictes du RGPD. L'audit a identifié les points de gouvernance suivants :
- **Minimisation des données :** L'API doit être configurée pour ne collecter et ne transmettre que le strict nécessaire au fonctionnement du service. Une révision des formulaires d'inscription est recommandée pour éviter la collecte de données non pertinentes.

- **Droit à l'effacement ("Droit à l'oubli") :** Une procédure automatisée doit être implémentée via un endpoint dédié côté API pour permettre la suppression définitive du compte et l'anonymisation des dossiers médicaux associés, conformément aux obligations légales.
- **Journalisation des accès :** Il est impératif de mettre en place une journalisation (audit log) de chaque accès aux données personnelles. Ce log doit permettre de savoir *qui* a accédé à *quelle* donnée et *quand*, sans pour autant stocker la donnée sensible elle-même dans le fichier de log.

4.2 Standard HIPAA (Health Insurance Portability and Accountability Act)

- Bien que d'origine américaine, le standard HIPAA définit les meilleures pratiques mondiales pour la gestion des données de santé (e-Health).
- **Chiffrement de bout en bout :** Toutes les données de santé (ePHI) doivent être chiffrées non seulement lors du transit (via TLS), mais également au repos dans la base de données de production.
- **Piste d'audit (Audit Trail) :** Une traçabilité inaltérable doit être assurée pour toutes les opérations de lecture et d'écriture sur les dossiers médicaux, permettant une reconstruction des événements en cas d'incident.
- **Gestion stricte des privilèges :** L'accès aux dossiers doit être régi par le principe du "besoin d'en connaître" (Need-to-know basis), limitant l'accès au personnel soignant autorisé uniquement pendant la durée du soin.
-

5. TESTS ET CONTRÔLES DE SÉCURITÉ RECOMMANDÉS

Afin de valider l'efficacité des mesures correctives et de maintenir un niveau de sécurité optimal, une campagne de tests rigoureuse doit être déployée.

- **Tests de pénétration API (OWASP API Top 10) :** Réalisation de tests d'intrusion ciblés pour identifier les vulnérabilités spécifiques aux APIs REST, notamment les défaillances d'autorisation au niveau de l'objet (BOLA), l'authentification brisée et l'exposition excessive de données. Ces tests doivent simuler des scénarios d'attaques réelles pour éprouver la robustesse du backend Laravel.
- **Analyse Statique de Sécurité (SAST) du code Laravel :** Utilisation d'outils d'analyse automatique pour détecter les mauvaises pratiques de codage, les

secrets en dur (hardcoded secrets) et les vecteurs d'injection potentiels avant même le déploiement en production.

- **Vérification de l'intégrité du stockage local Flutter** : Audit spécifique sur des terminaux physiques (Android et iOS) pour s'assurer que les tokens JWT et les données utilisateur ne sont pas accessibles via des outils de débogage ou sur des appareils "rootés"/"jailbreakés".
- **Simulation d'attaques automatisées** : Utilisation de suites d'outils professionnelles telles que **Burp Suite** ou **OWASP ZAP** pour automatiser la détection de failles courantes (Fuzzing, Broken Access Control) et tester la résistance des mécanismes de Rate Limiting.

6. RECOMMANDATIONS STRATÉGIQUES

La sécurité ne doit pas être un événement ponctuel mais un processus continu intégré au cycle de vie de l'application.

- **Intégration du DevSecOps dans le pipeline CI/CD** : Automatiser les contrôles de sécurité (SAST et DAST) à chaque "commit" ou "pull request". Cela permet de détecter les régressions de sécurité en temps réel et d'empêcher le déploiement de vulnérabilités connues.
- **Gestion proactive des dépendances** : Mise en œuvre de scans réguliers des packages Composer (Laravel) et Pub (Flutter) pour identifier les vulnérabilités critiques dans les bibliothèques tierces. Un plan de mise à jour rapide doit être défini pour réagir aux CVE (Common Vulnerabilities and Exposures) publiées.
- **Programmes de formation et sensibilisation** : Organisation de sessions de formation régulières pour les équipes de développement sur les principes de l'**API Security**. La sensibilisation au codage sécurisé est le rempart le plus efficace contre les erreurs humaines.
- **Plan de Réponse aux Incidents (SIRT)** : Définition d'un protocole de crise clair en cas de fuite de données avérée ou de compromission massive de tokens. Ce plan doit inclure des procédures de révocation de sessions, des canaux de communication pour les utilisateurs impactés et des méthodes d'analyse post-mortem pour renforcer le système.

7. CONCLUSION

En conclusion, l'architecture technique de **QuickCare** présente une base technologique solide et moderne. Toutefois, l'analyse a mis en lumière des zones de vulnérabilité critiques qui nécessitent un renforcement immédiat de la gouvernance

des accès, une sécurisation accrue de la gestion des tokens et une réduction drastique de l'exposition des données médicales.

La mise en œuvre rigoureuse des recommandations détaillées dans ce rapport permettra d'atteindre les objectifs stratégiques suivants :

- **Réduction significative de la surface d'attaque :** En verrouillant les endpoints et en filtrant les flux, nous limitons drastiquement les opportunités pour un acteur malveillant
- **Mise en conformité réglementaire totale :** L'alignement sur les standards RGPD et HIPAA garantit la pérennité juridique de l'application et la protection des droits des patients.
- **Maturité accrue en matière de sécurité applicative :** En intégrant la sécurité dès la phase de conception (Security by Design), l'entreprise se dote d'une culture de résilience durable.
- L'adoption d'une posture de sécurité proactive est aujourd'hui l'atout majeur pour pérenniser la confiance des patients et assurer le succès de QuickCare dans un écosystème de santé de plus en plus numérisé et exposé.